Convolutional Neural Networks

- From pixels to features: what a convolution does
- Shapes, padding/stride/dilation
- receptive field
- Pooling
- A Convolutional Neural Network (CNN)
- Interpretability, pitfalls, and practical checklists

Why CNNs?

- Locality: nearby pixels are related
- Translation equivariance: features shift with objects. If a feature (like an cat) is detected in one part of an image, the filter will detect the same feature in the same way even if the cat is moved to a different position.
- Weight sharing: same filter applied across the image ⇒ far fewer params than MLP
- Vision: used in classification, detection, segmentation
- Also used in Audio/spectrograms, time–frequency signals
- Also used with Transformers (ConvNeXt)

Locality: nearby pixels are related

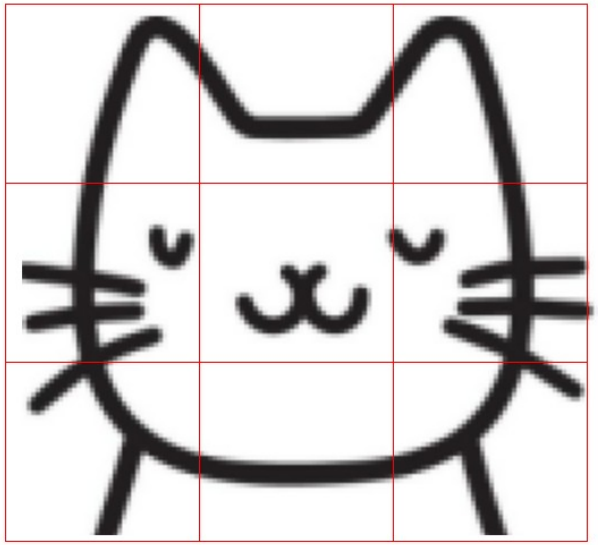
What is this a picture of?

Locality: nearby pixels are related

+

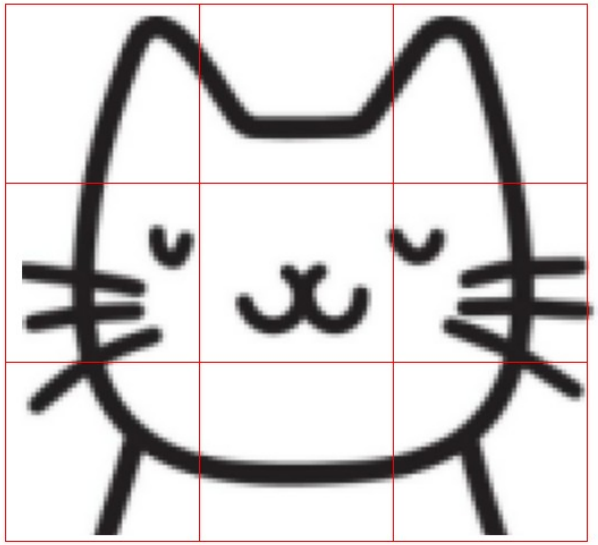
A fully connected Net (MLP) sees a picture this way
It has to figure out how pixels are spatially related

Locality: nearby pixels are related



A conv net sees it this way

Locality: nearby pixels are related

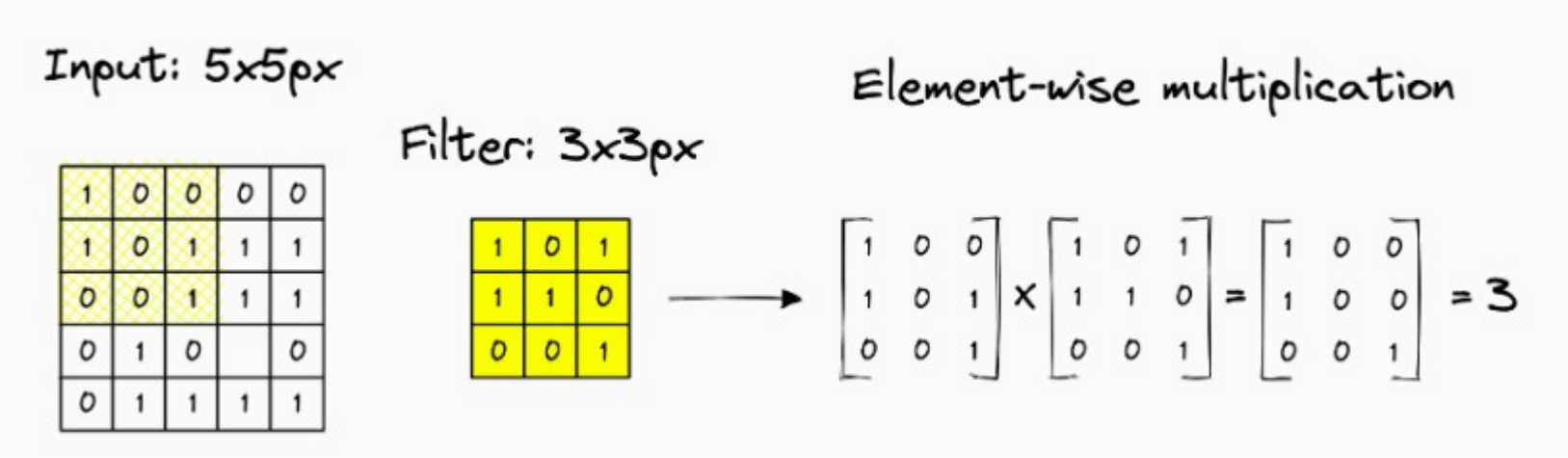


+

A conv net sees it this way
Notice that pixels are correctly ordered spatially

What is a Convolution?

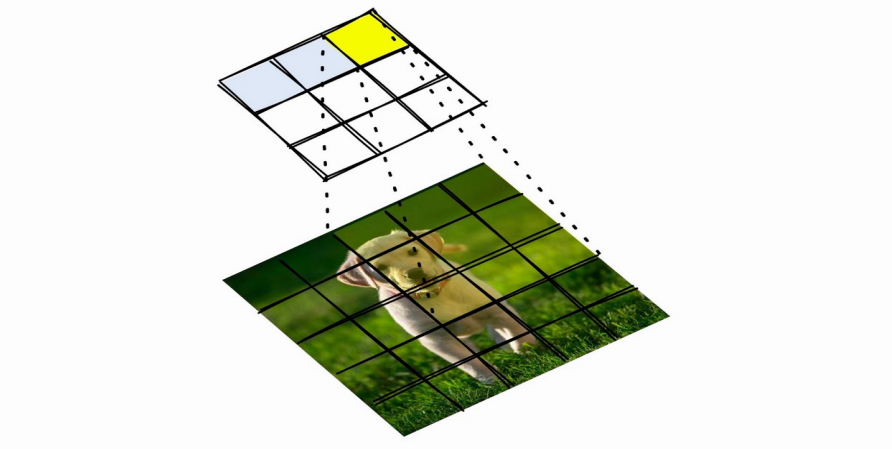
- Learned filter (kernel) slides spatially over input; outputs a ***feature map***
- The following shows a filter applied to upper left of an image. It produces the number 3 (will be part of final feature map)
- Trained to detect LOCAL features (edges, textures, etc)



From https://www.pinecone.io/learn/series/image-search/cnn/

What is a Convolution?

- This same filter slides across the entire image, from left to right and top to bottom to produce the 3x3 feature map shown below.

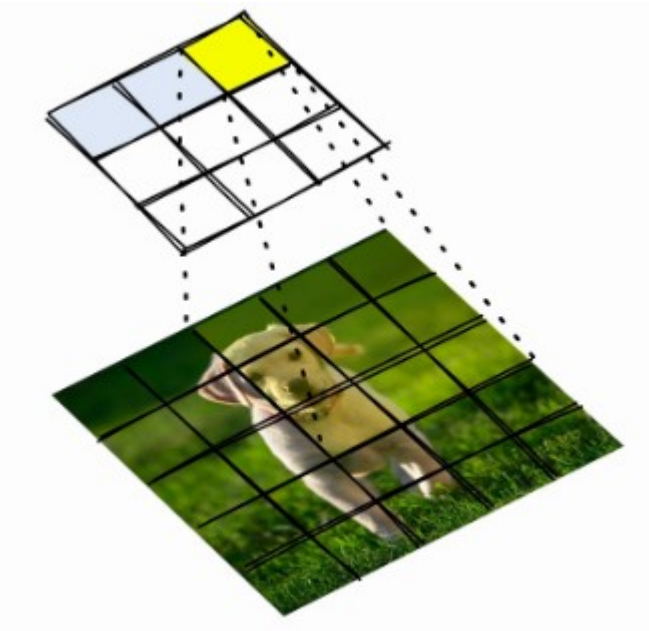


From https://www.pinecone.io/learn/series/image-search/cnn/

What is a Feature Map?

- A feature map is the 2D grid of activation values produced by applying one filter (kernel) across an input (image or previous layer). Think of it as the “response image” for a single pattern detector.

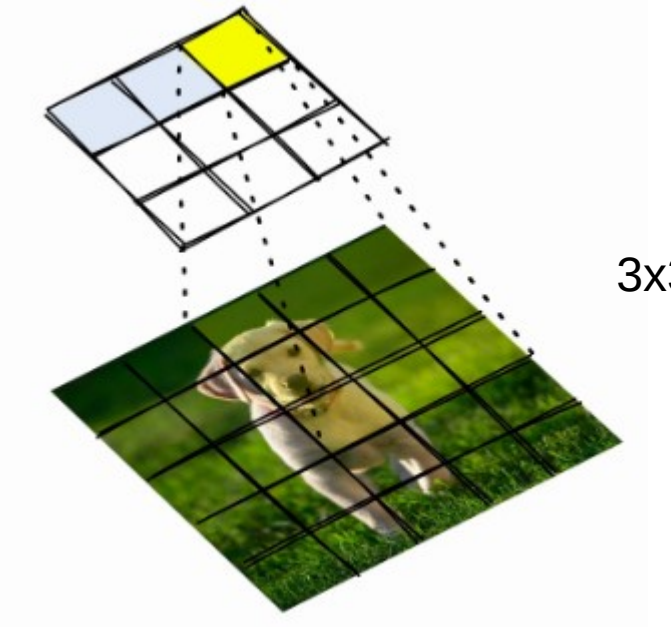
Feature Map



From https://www.pinecone.io/learn/series/image-search/cnn/

What is a Receptive Field

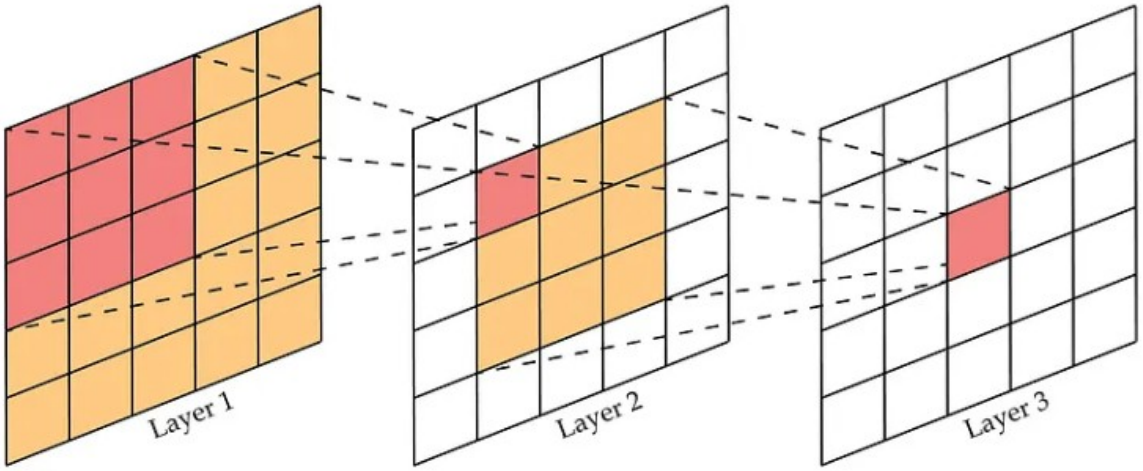
- Region of input a unit can “see”



3x3 Receptive Field

What is a Receptive Field

- Grows with depth, kernel size, and stride/dilation



The red square in Layer 2 sees the upper 3x3 red squares in Layer 1

But the red square in Layer 3 sees the 9 squares in Layer 2,
each of these was created by the 3x3 filter sliding across Layer 1

The red square in layer 3 effectively sees the entirety of Layer 1

Padding, Stride, Dilation

- Padding: Controls size of output volume by ‘picture framing’ image with empty pixels
- Stride: downsamples, larger step ⇒ smaller feature maps
- Dilation expands receptive field without extra params

Go to this Convolution Visualizer to see how these parameters work together

See https://ezyang.github.io/convolution-visualizer/index.html

Size of Output Feature Map

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{out}: number output features
n_{in} :number of input features
p: padding
k: filter size
s: stride

Size of Output Feature Map

Example:
Image size is 28*28*3
filter = 3x3
Padding=1
Stride=1

Feature Map size:
W2=H2=(28-3 +2*1)/1+1=28

The number learnable parameters:
Conv filter:
3*3*3 +1 = 28

If Image connected to a
single MLP instead:
28*28*3 +1=2353

The conv filter has many fewer parameters to learn

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

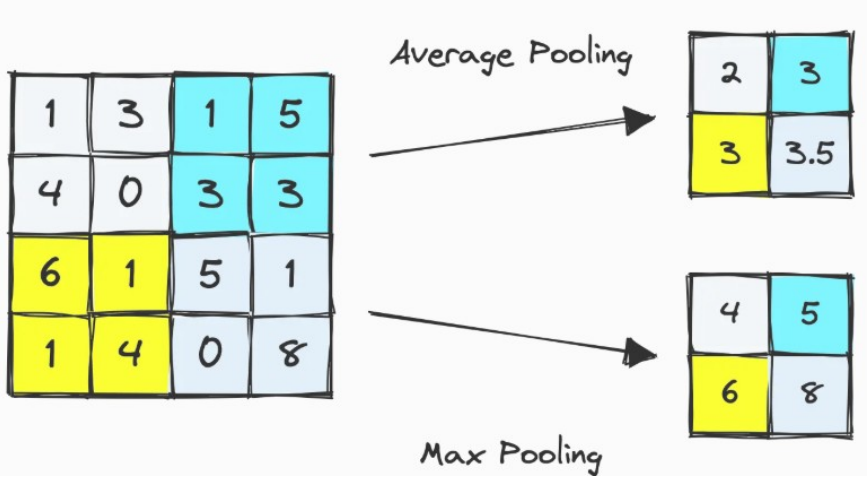
n_{out}: number output features
n_{in} :number of input features
p: padding
k: filter size
s: stride

Pooling

- A way to downsample a layer (and reduce its size)
- Information across several pixels is compressed into a single activation
- 2 common methods (although max is preferred)
- Final size formula is the same one used for feature maps

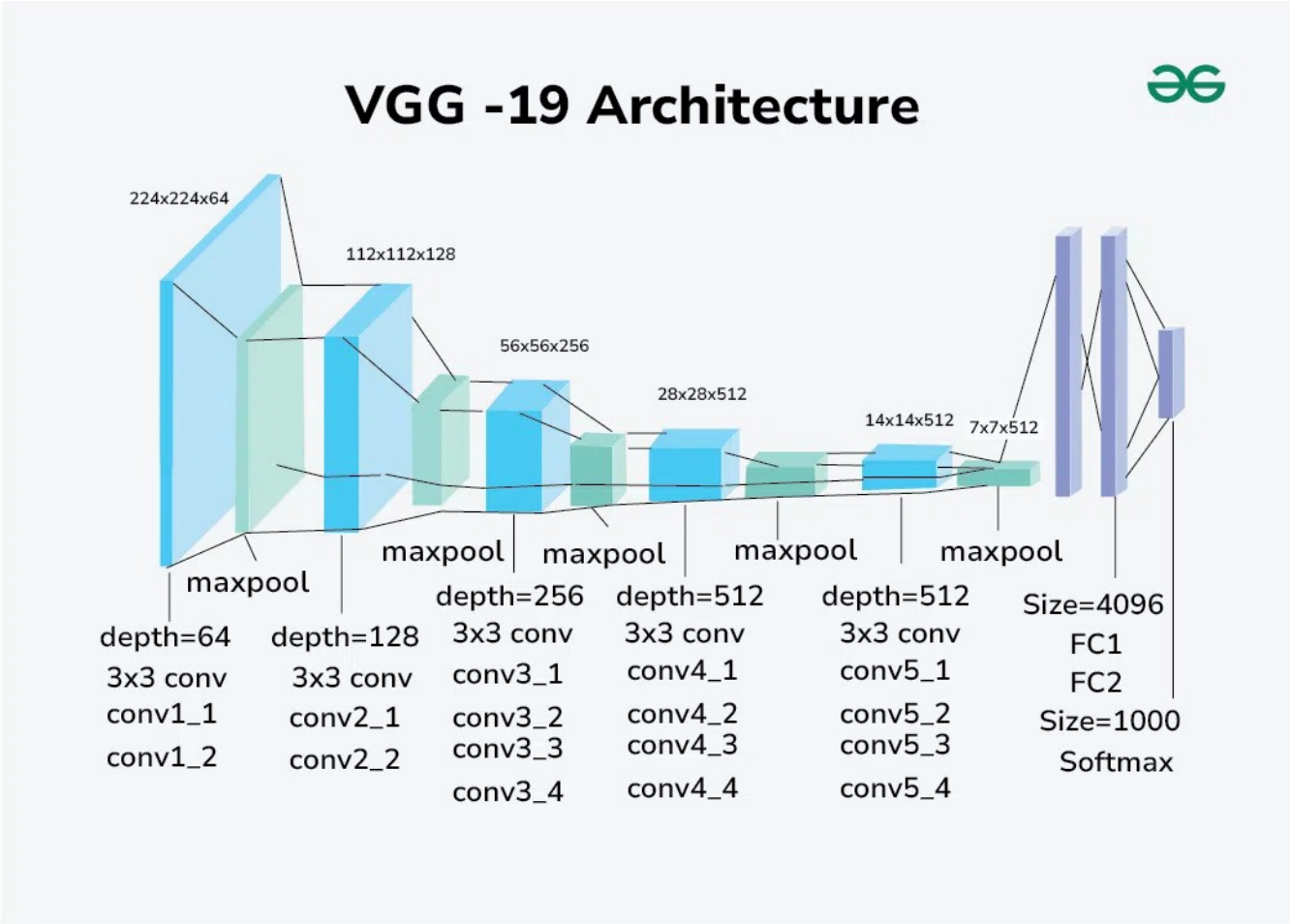
$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

- Ex. w=4, k=2, p=0, s=2 => (4-2)/2+1=2



From https://www.pinecone.io/learn/series/image-search/cnn/

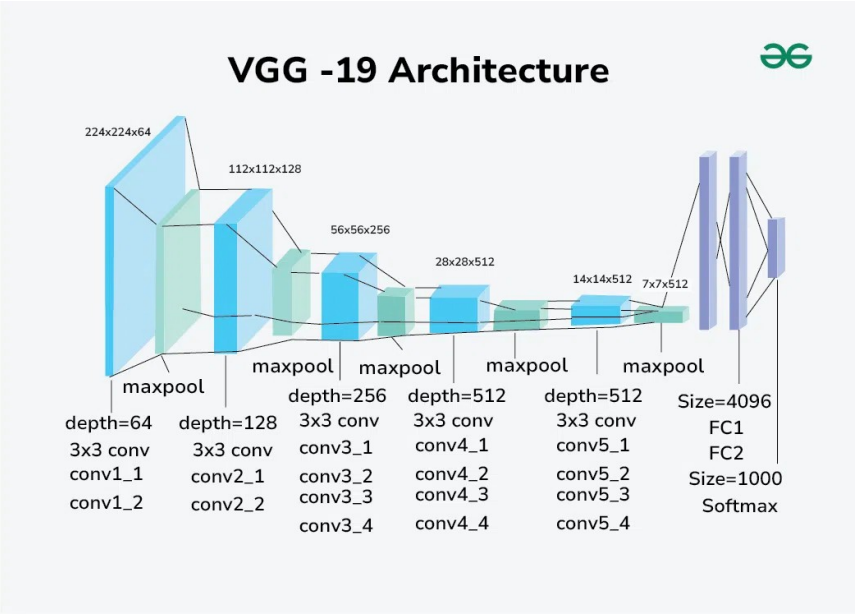
A Convolutional Neural Net



From https://www.geeksforgeeks.org/computer-vision/vgg-net-architecture-explained/

Fixed input size

- Conv layers work with <u>any</u> image size(Height x Width)
- **But, the fully connected (FC) layer at the end of the CNN ‘fixes’ the size of the input image**
- By flattening the feature map that feeds into the final FC layer
- Ex. the max pool layer below is flattened (7*7=49). Which fixes the input size to 224,224



From https://www.pinecone.io/learn/series/image-search/cnn/

Fixed input size Example

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,
dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)
class torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)
```

```
import torch, torch.nn as nn
model = nn.Sequential(
  nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.BatchNorm2d(16),
  nn.MaxPool2d(2),
  nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
  nn.MaxPool2d(2),
  nn.Flatten(),
  nn.Linear(32*16*16, 10)
)
print(model(x).shape)
```

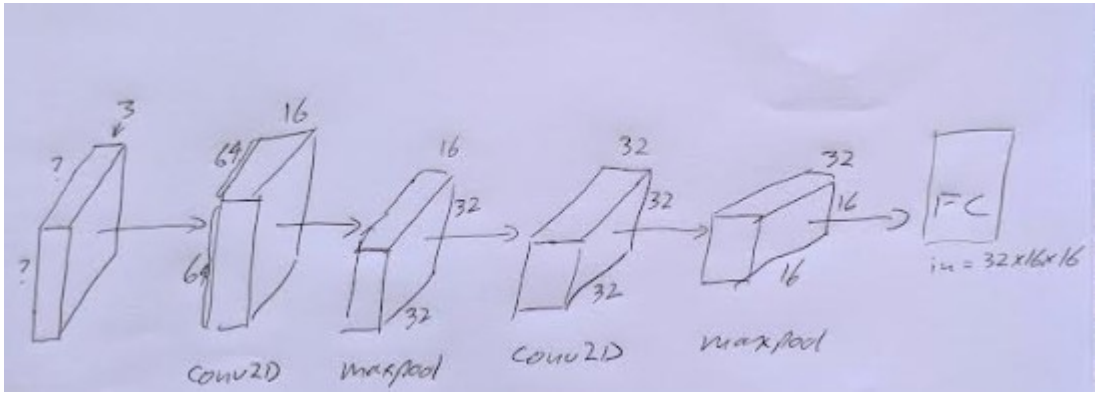
Fixed input size Example

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,
dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)
class torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)
```

```
import torch, torch.nn as nn
model = nn.Sequential(
  nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.BatchNorm2d(16),#f=3,p=1 preserves size
  nn.MaxPool2d(2),  # before 16x64x64 after 16x32x32
  nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),  #f=3,p=1 preserves size
  nn.MaxPool2d(2),  # before 32x32x32 after 32x16x16
  nn.Flatten(),
  nn.Linear(32*16*16, 10) #numbfilters * H * W
)
print(model(x).shape)
```

Model input size=3x64x64

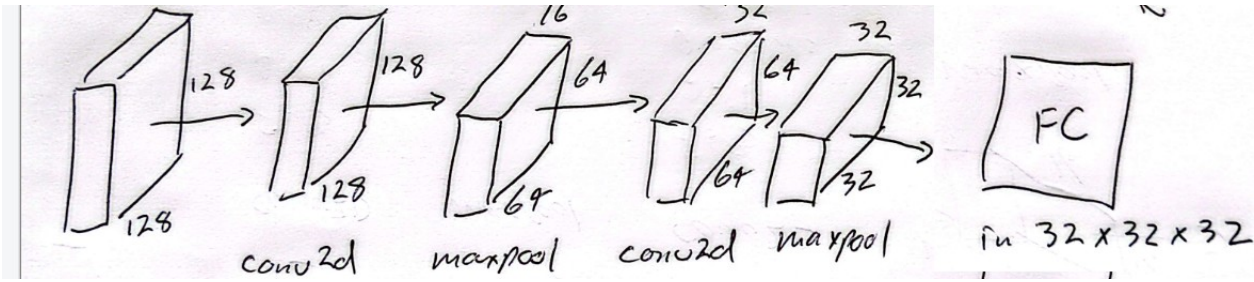
What if input size was 3x128x128? What changes?



Fixed input size Example

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,
dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)
class torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)
```

```
import torch, torch.nn as nn
model = nn.Sequential(
  nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.BatchNorm2d(16),#f=3,p=1 preserves size
  nn.MaxPool2d(2),  # before 16x64x64 after 16x32x32
  nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),  #f=3,p=1 preserves size
  nn.MaxPool2d(2),  # before 32x32x32 after 32x16x16
  nn.Flatten(),
  nn.Linear(32*16*16, 10) #numbfilters * H * W
)
print(model(x).shape)
```



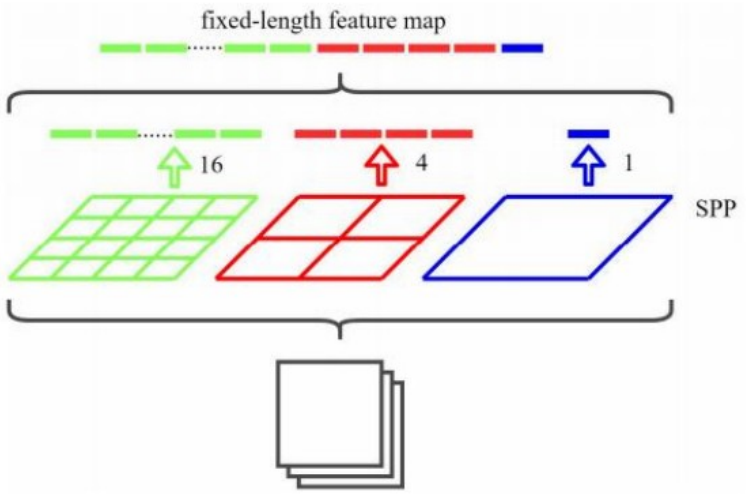
What if input size was 3x128x128? What changes?

FC has to be =32x32x32

Spatial Pooling – Any sized input

- Divides the variable-sized feature map into fixed-size spatial bins
- Then pools features within each bin to produce a fixed-length feature vector.
- All outputs concatenated, generating a constant sized input for fully connected layers (removes constant sized input requirement)

Ex. Feature map is 28x28
Green bin size (28/4)=> 7x7
Red bin siz (28/2)=> 14x14
Blue bin size=> 28x28
Fixed length feature map=16+4+1=21



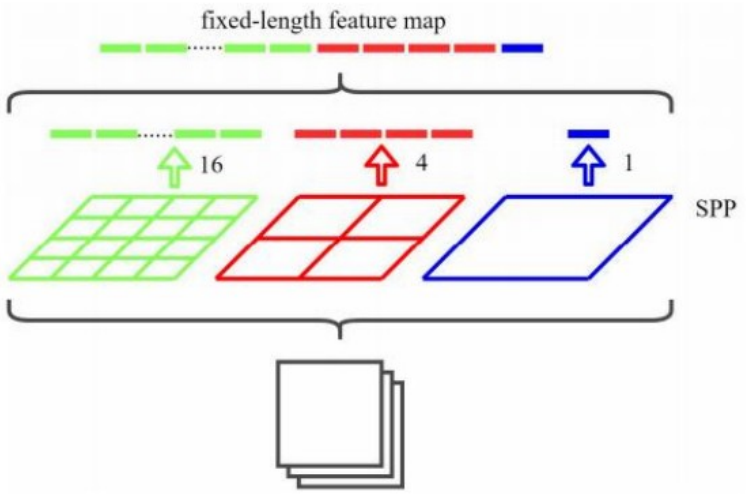
Spatial Pooling – Any sized input

- Divides the variable-sized feature map into fixed-size spatial bins
- Then pools features within each bin to produce a fixed-length feature vector.
- All outputs concatenated, generating a constant sized input for fully connected layers (removes constant sized input requirement)

Ex. Feature map is 28x28
Green bin size (28/4)=> 7x7
Red bin siz (28/2)=> 14x14
Blue bin size=> 28x28
Fixed length feature map=16+4+1=21

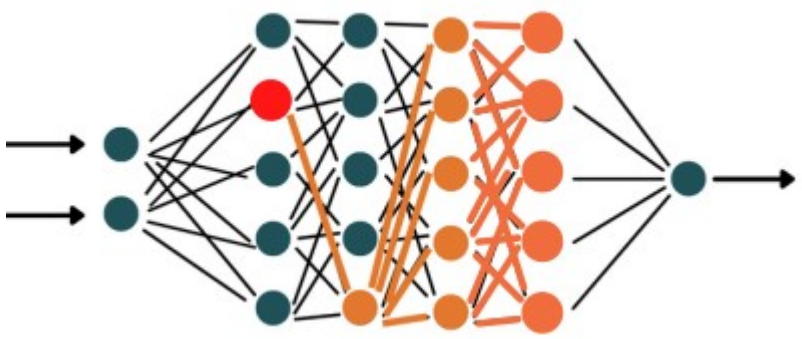
Ex. Feature map is 64x64
Green bin size (64/4)=> 16x16
Red bin size (64/2) => 32x32
Blue bin size=> 64x64
Fixed length feature map=16+4+1=21

Always the same length



Unstable Gradients

- Sometimes gradients become vanishingly small or so big that they become NaNs



Ex.
To get the gradient of the Red neuron. We backprop through all following layers. What if the gradients are small at the final layer? Red gradient becomes.

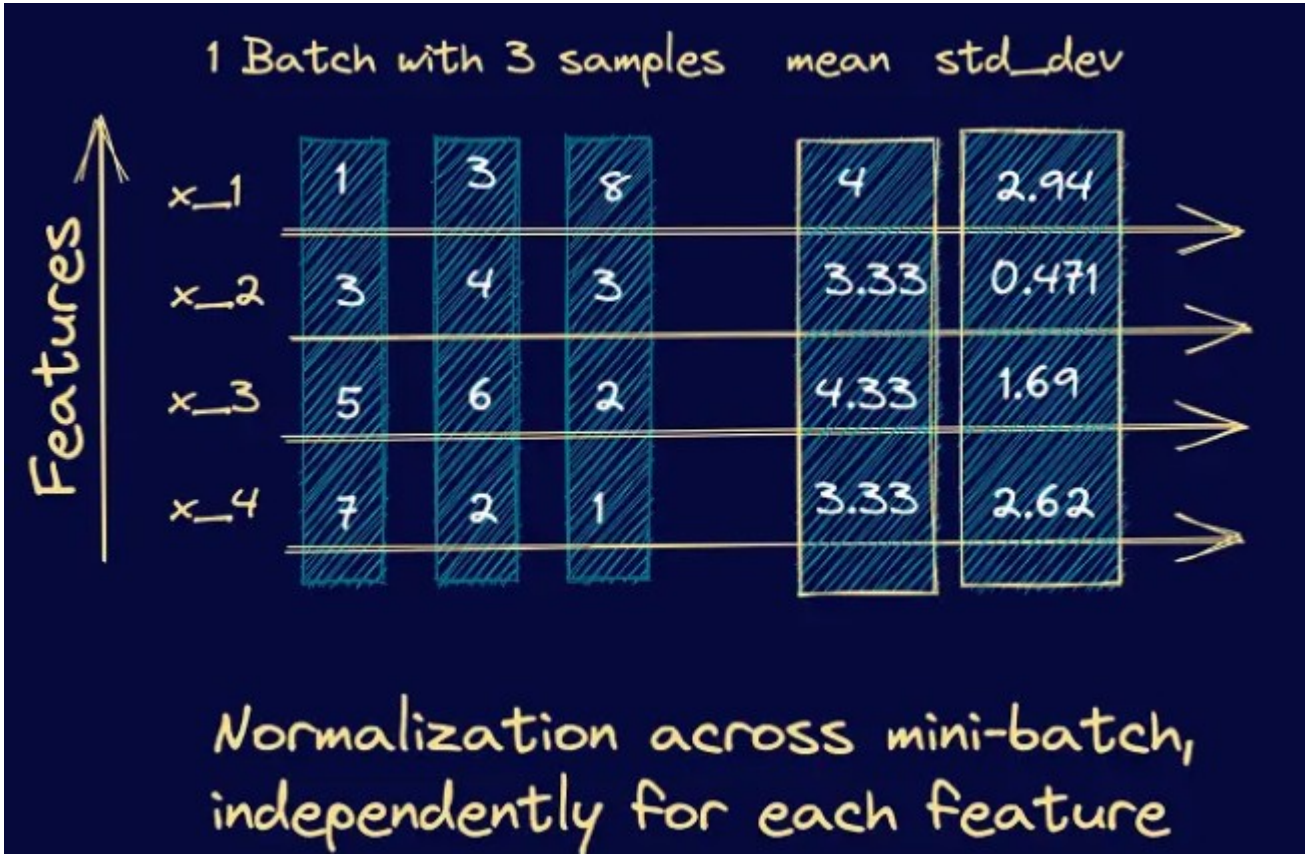
$$\text{small number} \times \text{small number} \times \dots \times \text{small number} = \text{very small number}$$

Batch normalization helps with this.

From https://www.youtube.com/watch?v=Jj_w_zOEu4M&t=183s

Batch Normalization

- Normalizes inputs so the mean=0, std=1

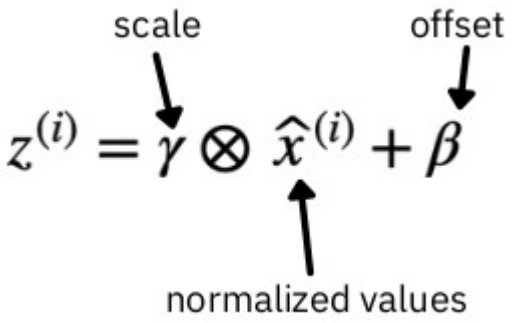


From https://www.pinecone.io/learn/batch-layer-normalization/

Batch Normalization

NNs are trained using mini batches of data. If mini batch distributions are very different (called internal covariate shift). Then the NN could be adjusting trainable params up for 1 batch and then back down for the next.

Solution: The following equation helps the NN to 'learn' how to adjust the normalized values so they match the overall data distribution.



Batch Normalization

Saves learnable params and an averaged sum of mean and standard deviation.

At inference. Use the saved params to normalize.

Advantages:

- Helps to prevent unstable gradients
- Can be used at the input to a NN. Eliminating the need to normalize input data
- NN tend toconverege fatser with batch normalization

Training Details that Matter

- Optimizers: AdamW or SGD+momentum; weight decay
- LR schedules: cosine, OneCycle; (later)
- Normalize inputs
- Careful splits so train and test sets are representative
- Data augmentation (image flip, crop, color shift etc…possibly coming later)

Transfer Learning (coming next)

- We are not going to train conv nets from scratch (takes way too long, tough to get details right)
- Instead we will fine tune existing models (there are a lot of them, sample below)

```
from torchvision import models
base = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in base.parameters(): p.requires_grad_(False)
base.fc = nn.Linear(base.fc.in_features, num_classes)
# Train head; then unfreeze last block with small LR
```

Evaluation & Interpretability

- Accuracy, confusion matrix, macro-F1
- Grad-CAM / saliency – these let you see what the CNN thought was important
- Failure case review – what model was most confidently wrong about, look at these images to see why.

Common Pitfalls & Debugging

- Shape/order mix-ups (NCHW vs NHWC)
- Overfitting small data (use Data augmentation to generate synthetic data)
- LR too high/low

Key Takeaways

- Convs = local, shared filters → efficient, fewer parameters than MLP
- Receptive Field /stride/dilation balance detail vs context